
Reference

Everything needed to regenerate or change the creative, plus the voiceover script.

Files

FILE	WHAT IT IS
<code>build.js</code>	Renders every asset from HTML. One command rebuilds the whole batch
<code>vo.py</code>	Generates the narration and measures it, so scene lengths match the read
<code>script.json</code>	Captions and voiceover lines in one place, so picture and read cannot drift
<code>vo-style.json</code>	The direction given to the TTS model per concept
<code>tokens.json</code>	The four Day 0 figures. Fill these and re-run
<code>vo-timing.json</code>	Measured scene lengths, written by <code>vo.py</code>
<code>VOICEOVER.md</code>	The recording script with in/out timecodes per line
<code>TTS-SETUP.md</code>	Which speech engines work, quota limits, and how to switch
<code>brand/</code>	The Kyntlo wordmark, full-colour and white knockout

Filling the Day 0 numbers

1. Run Day 0 — submit the enquiry forms, log the reply times. About two hours.
2. Put the results in `tokens.json`.
3. Rebuild:

```
NODE_PATH=/opt/node22/lib/node_modules node build.js
```

Every asset regenerates with real numbers in place of the amber slots.

Re-recording the voiceover

The narration is synthetic (Gemini TTS, voice Charon). To replace it with a human read, the picture is already cut to the line lengths — `VOICEOVER.md` has the in/out timecode for every line, so a natural read drops in without re-editing anything.

Concept A should be recorded by the founder. It speaks in the first person — *"I filled in the contact form on N Dubai businesses"* — and the campaign's whole argument is that the claim is literally true and independently checkable. A convincing synthetic voice makes that harder to defend rather than easier, because nobody listening can tell. It is sixty seconds of phone audio. Concept B makes no first-person claim, so synthetic narration suits it fine.

Changing copy

Edit `script.json` for the videos — it drives both the on-screen captions and the spoken lines, so they cannot fall out of sync. Then re-run `vo.py` and `build.js`. The per-line cache means only what changed gets re-synthesised.

The one lesson worth keeping

On Gemini TTS the **direction controls duration**, not just tone. The same line, same voice, same model:

DIRECTION	LENGTH
"...unhurried..."	6.12s
"...normal conversational speed, no dramatic pauses, this is a 25-second social video..."	3.60s

41% shorter, and across the script it was the difference between a 49-second TikTok ad and a 32-second one. Under 30 seconds is where short-form cold traffic performs, so the first version was unusable however good it sounded.

When editing `vo-style.json` : **state the format and the pace, not only the mood.**